Command-Based Robots

RamRodz Robotics · Team 6366 · Programming Worksheet — Lesson 7

RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS · SIMPSONVILLE, SC

🧠 Focus Check-In 😵 Struggling 😬 Okay 🥹 Ready

🔥 **TODAY'S BIG QUESTION**

What if pressing a button on the controller could automatically grab a ball, lift it, and drop it — all without you holding the button the whole time?

🅿️ **Parking Lot — Random Thoughts**

1 Key Words — Check Off When Explained

Command-Based
A WPILib programming style where robot actions are packaged as Command objects and managed by a Scheduler.

Command
An object that represents one robot action. It can start, run, and finish on its own timetable.

SubsystemBase
The WPILib class your subsystem *extends*. It connects your class to the Command Scheduler automatically.

Scheduler
WPILib's built-in manager that runs Commands at the right time and makes sure two commands don't fight over the same hardware.

runOnce()
A SubsystemBase helper that wraps one block of code into a Command that fires once, then finishes automatically.

Requirement
Tells the Scheduler which subsystem a command "owns." Only one command can own a subsystem at a time.

RobotContainer
The class in a Command-Based project where you create subsystems and wire controller buttons to commands.

Lambda ( ) ->
A compact way to write a small chunk of code that will be run later — used inside runOnce() and run().

2 The Old Way vs. The Command Way

Before Command-Based, you called subsystem methods directly inside a loop every robot tick. That works for simple robots — but bigger robots have many subsystems competing for the driver's attention. Command-Based solves this.

Old Style (Lesson 5 & 6)	Command-Based (Lesson 7+)
Methods *do the action directly* when called	Methods *return a Command* — the Scheduler decides when to run it
You manage timing with `if` checks every loop	Commands manage their own timing — they finish on their own
If two actions conflict, bad things happen	Scheduler prevents two commands from fighting over the same motor
`intake.run();` — runs immediately	`controller.a().onTrue(intake.grab());` — runs when A is pressed

- [ ] I understand that Command-Based means methods *return* Commands instead of doing actions directly.

✏️ Quick Question — In the old style, when does the intake run? In Command-Based, when does it run?

3 SubsystemBase — The New Foundation

In Lesson 6 your subsystem was a plain Java class. In Command-Based projects, your subsystem **extends SubsystemBase** — that one word unlocks the Scheduler, requirements, and the helper methods you'll use all season.

Java — SubsystemBase skeleton

```
// Import WPILib's SubsystemBase
import edu.wpi.first.wpilibj2.command.SubsystemBase;
import edu.wpi.first.wpilibj2.command.Command;

public class ExampleSubsystem extends SubsystemBase {

    // fields go here — same as before

    public ExampleSubsystem() {
        // constructor — same as before
    }

    // periodic() runs every robot tick (20 ms) — optional
    @Override
    public void periodic() {
        // update dashboard readouts, check sensors, etc.
    }

    // methods now RETURN Command objects instead of doing actions directly
    public Command doSomething() {
        return runOnce(() -> { /* action here */ });
    }
}
```

Plain English: **extends SubsystemBase** means "my class inherits everything WPILib's SubsystemBase already knows how to do." You get `runOnce()`, `run()`, and automatic Scheduler registration for free.

- [ ] I see that the class declaration now says `extends SubsystemBase` and methods return `Command` instead of `void`.

✏️ **Fill In — What return type do command-returning methods use? What keyword connects the subsystem to SubsystemBase?**

4 runOnce() and the Lambda Arrow

λ New Syntax: The Lambda `() ->`

A lambda is a mini code-block written inline. `() -> { … }` means *"here is a chunk of code — run it when you're ready."*
You've been writing full methods with names. A lambda is the same idea but nameless, passed directly to `runOnce()`.

Java — runOnce() anatomy

```
public Command grab() {
    return runOnce( () -> {          // ① return a Command  ② lambda starts here
        grabMotor.set(0.7);             // ③ code that runs once when commanded
        isGripping = true;            // ④ update state
    });                                 // ⑤ lambda ends — 'this' subsystem auto-requ
}
```

Plain English: **runOnce(() -> { … })** packages the code inside the braces into a one-shot Command. When the Scheduler fires it, the code runs exactly once, then the command finishes automatically. Because it's called on *this* subsystem, the Scheduler knows only this subsystem's commands can run at the same time.

runOnce() vs run()

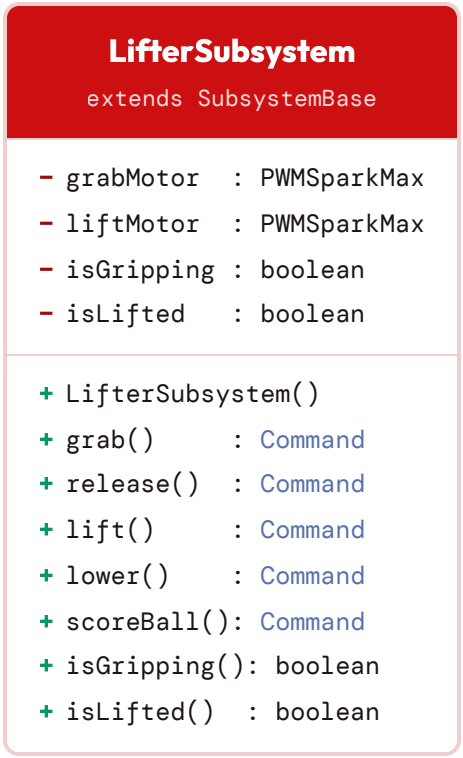
`runOnce()` — fires the lambda once, then done. Use for instant actions: open a claw, set a motor speed, flip a solenoid.
`run()` — keeps firing the lambda every tick until interrupted. Use for continuous actions: hold a wheel spinning, track a target.

- [ ] I can read a `runOnce( () -> { … } )` block and explain what the lambda does.

Predict It — If you wrote `run(() -> { liftMotor.set(0.8); })`, would the motor run once or keep running? Why?

5 Building LifterSubsystem — Step by Step

We're building a subsystem for a robot that can grab a ball, lift it, and release it. Study the UML first, then follow along with each step.



Notice: Every method that controls the robot now returns `Command` (shown in blue in the UML). Only the two *state getters* at the bottom still return `boolean` — they just read a value, they don't control anything.

STEP A — FIELDS & CONSTRUCTOR

Same as Lesson 6 — declare private fields, create motors in the constructor.

Java — Fields & Constructor

```
public class LifterSubsystem extends SubsystemBase {

    private final PWMSparkMax grabMotor;   // motor that grips the ball
    private final PWMSparkMax liftMotor;   // motor that raises the arm
    private boolean isGripping;             // are we gripping right now?
    private boolean isLifted;               // is the arm raised?

    public LifterSubsystem() {
        grabMotor  = new PWMSparkMax(0);  // grip motor on PWM port 0
        liftMotor  = new PWMSparkMax(1);  // lift motor on PWM port 1
        isGripping = false;
        isLifted   = false;
    }
```

Plain English: **final** on the motor fields means "once created in the constructor, this variable never points to a different motor." Good practice — motors don't get swapped mid-match.

STEP B — GRAB() AND RELEASE()

Both use `runOnce()` — the motor fires once and the command finishes.

Java — grab() and release()

```
    public Command grab() {
        return runOnce(() -> {
            grabMotor.set(0.7);   // spin inward at 70% to grip ball
            isGripping = true;  // remember we're gripping
        });
    }

    public Command release() {
        return runOnce(() -> {
            grabMotor.set(-0.4); // spin outward at 40% to drop ball
            isGripping = false; // no longer gripping
        });
    }
```

Plain English: **grab()** and **release()** look almost identical — they both package a motor call into a runOnce Command. The only difference is the direction and speed. Notice the return type is `Command`, not `void`.

STEP C — LIFT() AND LOWER()

Same pattern for the lift arm.

Java — lift() and lower()

```
public Command lift() {
    return runOnce(() -> {
        liftMotor.set(0.8);   // drive arm upward at 80%
        isLifted = true;     // remember arm is raised
    });
}

public Command lower() {
    return runOnce(() -> {
        liftMotor.set(-0.6);  // drive arm downward at 60%
        isLifted = false;     // arm is back down
    });
}
```

STEP D — SCOREBALL() SEQUENCE COMMAND

This is where Command-Based really shines — chain multiple commands together automatically.

Java — scoreBall() with Commands.sequence()

```
public Command scoreBall() {
    return Commands.sequence(
        grab(),                        // step 1: grip the ball
        Commands.waitSeconds(0.5),   // pause 0.5 s for grip to close
        lift(),                        // step 2: raise the arm
        Commands.waitSeconds(1.0),   // pause 1 s to reach height
        release()                     // step 3: drop the ball
    );
}
```

💬 Plain English: **Commands.sequence()** is like a to-do list — it runs each command in order, waiting for one to finish before starting the next. Press one button, the robot handles the whole grab-lift-release routine by itself.

Java — State Getters (no Command needed)

```
    public boolean isGripping() { return isGripping; }
    public boolean isLifted()   { return isLifted;   }
}
```

💬 Plain English: These two getters just *read* a boolean — they don't tell the robot to do anything. A method that only reads doesn't need to return a Command.

- [ ] I followed all five steps and I can explain why `grab()` returns a `Command` but `isGripping()` returns a `boolean`.

✏️ **In Your Own Words — What does `Commands.sequence()` do that a single `runOnce()` can't?**

🎯 Robot Scenario — Wire the Lifter to a Controller

🏆 Your Robot at Competition

Your robot has a working `LifterSubsystem`. Now you need to wire it up in `RobotContainer` so the driver can use it. Here's the button map the drive team wants:

- **A button** → grab the ball
- **B button** → release the ball
- **Right Bumper** → lift the arm
- **Left Bumper** → lower the arm
- **X button** → run the full `scoreBall()` sequence

Java — RobotContainer wiring pattern

```
public class RobotContainer {

    // Create the subsystem and controller — both are fields
    private final XboxController controller = new XboxController(0);
    private final LifterSubsystem lifter    = new LifterSubsystem();

    public RobotContainer() {
        configureBindings();     // call the wiring method
    }

    private void configureBindings() {
        controller.a().onTrue( lifter.grab() );          // A → grab
        controller.b().onTrue( lifter.release() );       // B → release
        controller.rightBumper().onTrue( lifter.lift() );  // RB → lift arm
        controller.leftBumper().onTrue( lifter.lower() ); // LB → lower arm
        controller.x().onTrue( lifter.scoreBall() );     // X → full sequence
    }
}
```

Plain English: **controller.a().onTrue(…)** means "when the A button is first pressed, schedule this command." The `onTrue()` binding runs the command once on press — you don't have to hold A while the robot does the whole sequence.

Design It — The coach wants to add a HOLD trigger: while the driver holds the Y button, keep the grab motor running. Would you use `runOnce()` or `run()`? What binding — `onTrue()` or something else?

Extend It — Add a fifth binding: when the driver presses the Start button, lower the arm AND release the ball at the same time (hint: Commands.sequence, or think about what two commands run).

- [ ] I can read the RobotContainer wiring and explain which method call triggers each command.

✅ End of Lesson — I Can...

- [ ] Explain why Command-Based is different from calling subsystem methods directly.
- [ ] Write a subsystem class that `extends SubsystemBase` with methods that return `Command`.
- [ ] Use `runOnce(() -> { … })` to package an action into a one-shot Command.
- [ ] Use `Commands.sequence()` to chain multiple commands into one automated routine.
- [ ] Wire subsystem commands to controller buttons in `RobotContainer` using `.onTrue()`.